UNCONSTRAINED OPTIMIZATION

TOPIC : APPLICATION OF STEEPEST DESCENT AND QUASI NEWTON METHODS

FIELD OF STUDY : BIOSCIENCE AND HEALTHCARE

TEAM MEMBERS

SHRESHTHA SENGUPTA
ANKITA DEY
KHUSHI SINGH
ARKADYUTI PAUL
SOUMYADEEP DAS
SHREEJA BANERJEE

DATA DESCRIPTION

- ★ We have selected the **Gallstone dataset from the UCI Machine Learning Repository.**
- ★ The clinical dataset was collected from the Internal Medicine Outpatient Clinic of Ankara VM Medical Park Hospital and includes data from **319 individuals** (June 2022–June 2023), **161 of whom were diagnosed with gallstone disease.**
- ★ It contains **38 features** including demographic, bioimpedance, and laboratory data, and was ethically approved by the Ankara City Hospital Ethics Committee.
- ★ The dataset is complete, with **no missing values**, and **balanced in terms of disease status,** eliminating the need for additional preprocessing.

Citation : Esen, I., Arslan, H., Aktürk, S., Gülşen, M., Kültekin, N., & Özdemir, O. (2024). Gallstone [Dataset]. UCI Machine Learning Repository.

Here is the link of the dataset!
https://archive.ics.uci.edu/dataset/1150/gallstone-1

We are interested in predicting whether an individual is diagnosed with gallstone disease.

Response variable
Gallstone Status(Y):
Gallstones present(0) and absent(1)

What model should be used for prediction?

- We observe that the target variable is binary in nature and we have a total of 38 features.
- This is a Binary Classification problem.
- We select a subset of most influential features for developing the model for prediction which is chosen on the basis of feature importance scores derived after fitting a Decision Tree Classifier to the entire dataset.
- Thus, out of 38, we obtain a total of 30 most influential features.

We want to predict the presence of Gallstone in the patient but the labelling of the target variable is inverted,hence we reverse the labelling.

Response variable:

Gallstone Status(Y): Gallstones present(1) and absent(0)

Top 10 influential features...

Feature	Importance
C-Reactive Protein (CRP)	0.300481
Vitamin D	0.090789
Bone Mass (BM)	0.057739
Body Mass Index (BMI)	0.043001
Alkaline Phosphatase (ALP)	0.039460
Extracellular Water (ECW)	0.034140
Hyperlipidaemia	0.032109
Obesity (%)	0.030933
High Density Lipoprotein (HDL)	0.030567
Extracellular Fluid/Total Body Water (ECF/TBW)	0.029064

What model should be used for prediction?

We thus proceed to fit a Multiple Logistic Regression Model for prediction. It is a supervised machine learning algorithm used for classification problems which predicts the probability that an observation belongs to a specific class of the target. It is used for binary classification where the output can be one of two possible categories such as Present(1) and Absent(0).

The logit models the probability of an observation belonging to a positive class is:

$$P\left(Y_i = 1 \mid x_1, x_2, \ldots\ldots x_m \right) = \frac{1}{1 + e^{-\left(\beta_0 + \beta_1 x_1 + \beta_2 x_2 \ldots + \beta_m x_m\right)}}$$

m : no. of features, here 30

The likelihood of predicting a positive class:

$$l(\beta) = \prod_{i=1}^{n} p_i^{Y_i} \left(1 - p_i\right)^{1 - Y_i}$$

$$y_i = \begin{cases} 1 & \text{if gallstone is present for observation } i \\ 0 & \text{if gallstone is absent for observation } i \end{cases}$$

$\beta^T = \left(\beta_0, \beta_1, \ldots\ldots, \beta_m\right)$ *is the parameter vector including intercept* β_0

The log-likelihood is thus given as:

$$l(\beta) = \sum_{i=1}^{n} \left[Y_i \log\left(p_i\right) + \left(1 - Y_i\right) \log\left(1 - p_i\right) \right]$$

$$\mathbf{x}_i = (x_{1i}, x_{2i}, \ldots, x_{mi})^T$$

i=1,2,…,319

Objective Function of our Optimization Problem

The optimization problem formulation to minimize the negative log-likelihood function in multiple logistic regression , using unconstrained optimization algorithms, can be written as:

$$\min_{\beta} \; -l(\beta) = \min_{\beta} \; -\sum_{i=1}^{n} \left[Y_i \log(p_i) + (1 - Y_i) \log(1 - p_i) \right]$$

Where $$p_i = \frac{1}{1 + e^{-\beta^T x_i}}$$

and

$$\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2, \ldots, \beta_m)^T$$

Is the parameter vector to be optimized.

$$\mathbf{x}_i = (x_{1i}, x_{2i}, \ldots, x_{mi})^T$$

Is the augmented predictor vector for observation i

In order to obtain the parameter estimates, we have to minimise the negative log likelihood function for which we use 2 Unconstrained Optimization Algorithms.

- Steepest Descent Method
- Quasi-Newton Method (BFGS Method)

The negative log likelihood function that is being minimised in logistic regression is a convex function with respect to the model parameters.The Hessian of this objective function is always positive semi-definite for all model parameters,hence confirming convexity. This is a convex optimization problem.

Steepest Descent Method

The Steepest Descent Method is an iterative optimization technique used to find the minimum of a differentiable function. At each iteration, the method moves the current solution in the direction opposite to the gradient of the function, since the gradient points in the direction of the steepest increase.

Mathematically the update rule is:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Where

- x_k = current point(vector of parameters),
- $\nabla f(x_k)$ = gradient of the objective function at x_k,
- α_k = step size or learning rate.

We update the alpha using the Armijo-Goldstein method. This method involves starting with a relatively large estimate of the step size for movement along the line search direction, and iteratively shrinking the step size (i.e., "backtracking") until a decrease of the objective function is observed that adequately corresponds to the amount of decrease that is expected, based on the step size and the local gradient of the objective function.

Quasi-Newton Method

The Quasi-Newton method is an optimization technique used to find the minimum or maximum of a function. It improves upon gradient descent by using an approximation of the Hessian matrix (second derivatives) instead of calculating it exactly, which can be expensive or difficult.

- Key idea: Use gradient information from previous steps to build an estimate of the function's curvature.
- Update rule (general form):

$$x_{k+1} = x_k - \alpha_k B_k \nabla f(x_k)$$

Where B_k approximates the inverse Hessian and α_k is a step size.
We specifically use the BFGS method wherein the B_k matrix is the approximation of the Hessian matrix instead of its inverse.

UPDATE RULE:

$$x_{k+1} = x_k - B_k^{-1} \nabla f(x_k)$$

The BFGS update formula for B_{k+1} uses the information from the change in the variable $s_k = x_{k+1} - x_k$

And the change in the gradient $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$

To refine the approximation:

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

Variable Description

1. C-Reactive Protein (CRP)
2. Vitamin D
3. Bone Mass (BM)
4. Body Mass Index (BMI)
5. Alkaline Phosphatase (ALP)
6. Extracellular Water (ECW)
7. Hyperlipidemia
8. Obesity (%)
9. High Density Lipoprotein (HDL)
10. Extracellular Fluid/Total Body Water (ECF/TBW)
11. Aspartate Aminotransferase (AST)
12. Body Protein Content (Protein) (%)
13. Visceral Muscle Area (VMA) (Kg)
14. Lean Mass (LM) (%)
15. Total Body Water (TBW)
16. Creatinine
17. Visceral Fat Area (VFA)
18. Hemoglobin (HGB)
19. Muscle Mass (MM)
20. Hepatic Fat Accumulation (HFA)
21. Total Cholesterol (TC)
22. Visceral Fat Rating (VFR)
23. Height
24. Age
25. Total Fat Content (TFC)
26. Weight
27. Low Density Lipoprotein (LDL)
28. Alanine Aminotransferase (ALT)
29. Total Body Fat Ratio (TBFR) (%)
30. Intracellular Water (ICW)

We have used 80 (train-set) percent of the data to estimate the parameters to solve the problem also known as “training” the model and the the rest 20 (test-set) percent is used to evaluate how well the model performs.

STEEPEST DESCENT (USER DEFINED)

1.Initial Values

$$\alpha = 0.1$$
$$\beta_0 = \beta_1 = \ldots\ldots = \beta_{30} = 0$$
$$Tolerance = 10^{-6}$$
$$Max_{Iteration} = 10000$$

The Control parameter(C) of the Armijo condition for updation of alpha is 10^(-4).
In our case, this is getting satisfied at the first step hence the alpha=0.1 remains same.

2. Final Parameter Estimates

Parameter	Symbol	Value
Intercept	β_0 (Bias)	**-0.44229746**
Weight 1	β_1	-1.94503265
Weight 2	β_2	0.77647383
Weight 3	β_3	1.47847172
Weight 4	β_4	0.81360786
Weight 5	β_5	-0.18653548
Weight 6	β_6	0.16437189
Weight 7	β_7	-1.28396996
Weight 8	β_8	-0.07882014
Weight 9	β_9	-0.28644254
Weight 10	β_{10}	0.68410777
Weight 11	β_{11}	1.42693205
Weight 12	β_{12}	0.40493313
Weight 13	β_{13}	-0.08975756
Weight 14	β_{14}	0.10413678
Weight 15	β_{15}	-0.67192574

Parameter	Symbol	Value
Weight 16	β_{16}	-0.04015077
Weight 17	β_{17}	-0.42243605
Weight 18	β_{18}	0.46879070
Weight 19	β_{19}	0.46030233
Weight 20	β_{20}	-0.25854153
Weight 21	β_{21}	-0.09252581
Weight 22	β_{22}	-0.06960762
Weight 23	β_{23}	-0.26944787
Weight 24	β_{24}	-0.31856009
Weight 25	β_{25}	0.74616638
Weight 26	β_{26}	-0.06444064
Weight 27	β_{27}	0.12199959
Weight 28	β_{28}	-1.02560462
Weight 29	β_{29}	-1.00049078
Weight 30	β_{30}	-1.43307885

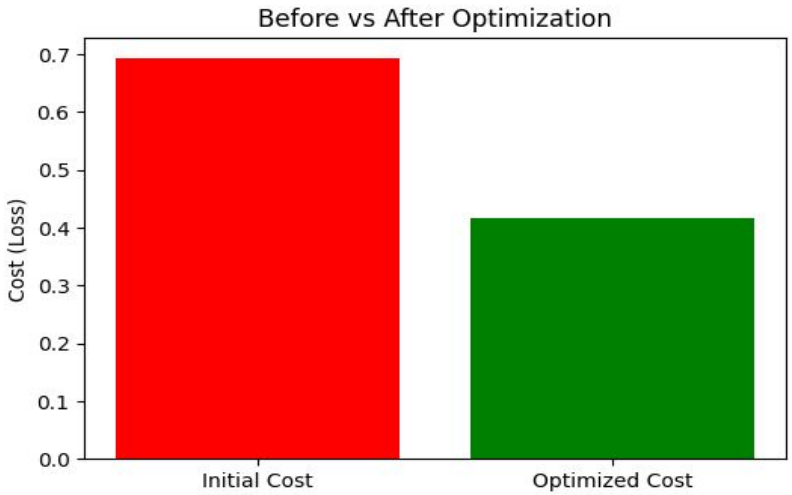
Evaluation on Test Data and Cost Reduction

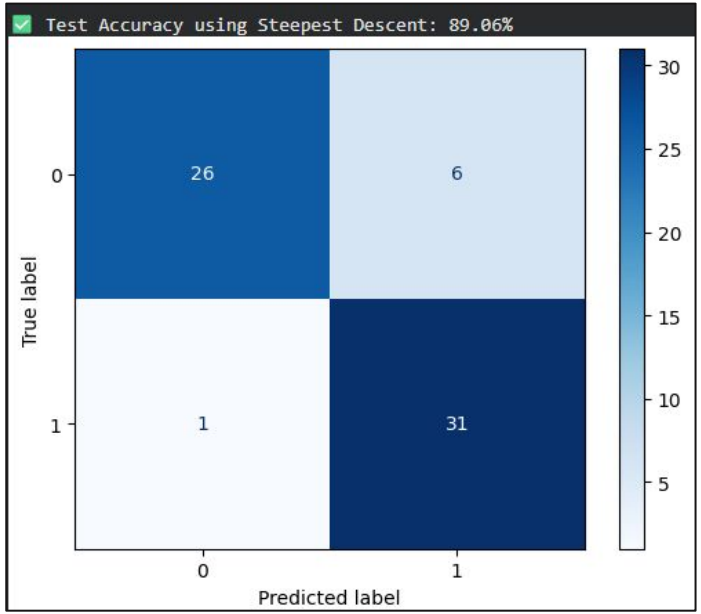
Cost Comparison:

	Metric	Value
0	Initial Cost	0.693147
1	Optimized Cost	0.416534
2	Cost Reduction	0.276613

We are using the mean of the predicted probabilities as the threshold.

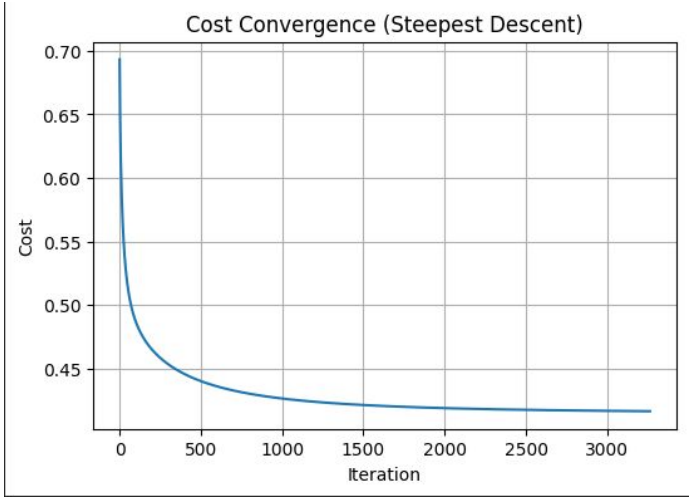
Test Accuracy using Steepest Descent: 89.06%





Convergence Table

	Iteration	Cost	Alpha	Grad_Norm
0	0	0.693147	NaN	NaN
1	1	0.674797	0.1	0.440821
2	2	0.660128	0.1	0.392939
3	3	0.648099	0.1	0.354763
4	4	0.637989	0.1	0.324322
3258	3258	0.416539	0.1	0.0031
3259	3259	0.416538	0.1	0.0031
3260	3260	0.416537	0.1	0.0031
3261	3261	0.416536	0.1	0.0031
3262	3262	0.416535	0.1	0.0031
3263	3263	0.416534	0.1	0.0031



The optimization algorithm converges for iteration number 3263.
The alpha remains 0.1 due to Armijo-Goldstein condition being satisfied.

BFGS Method (User Defined)

1.Initial Values

$$\beta_0 = \beta_1 = \ldots\ldots = \beta_{30} = 0$$

$$Max_{Iteration} = 1000$$

$$Tolerance = 10^{-5}$$

2. Final Parameter Estimates

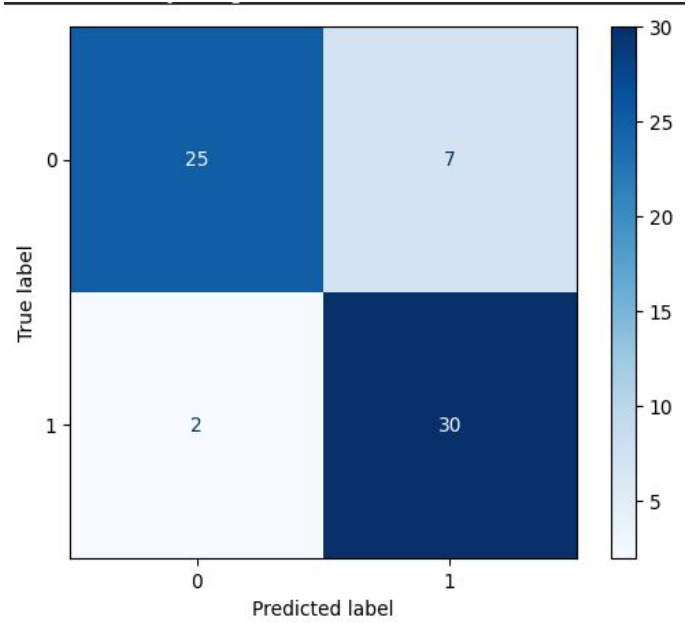
Parameter	Symbol	Value
Intercept	β_0 (Bias)	**-0.61241686**
Weight 1	β_1	-2.02727332
Weight 2	β_2	0.81504458
Weight 3	β_3	1.73517096
Weight 4	β_4	2.10691349
Weight 5	β_5	-0.22056934
Weight 6	β_6	0.94617558
Weight 7	β_7	-2.23441696
Weight 8	β_8	-0.08528783
Weight 9	β_9	-0.30560831
Weight 10	β_{10}	0.39152825
Weight 11	β_{11}	1.53775412
Weight 12	β_{12}	0.56869619
Weight 13	β_{13}	-0.07944537
Weight 14	β_{14}	-0.85393755
Weight 15	β_{15}	-0.82014875

Parameter	Symbol	Value
Weight 16	β_{16}	-0.03809811
Weight 17	β_{17}	-0.19316010
Weight 18	β_{18}	0.49853840
Weight 19	β_{19}	0.66186341
Weight 20	β_{20}	-0.27568773
Weight 21	β_{21}	-0.03429643
Weight 22	β_{22}	-0.17687947
Weight 23	β_{23}	0.51831466
Weight 24	β_{24}	-0.31101973
Weight 25	β_{25}	1.44449643
Weight 26	β_{26}	-2.82333046
Weight 27	β_{27}	0.04962031
Weight 28	β_{28}	-1.07901656
Weight 29	β_{29}	-1.82153711
Weight 30	β_{30}	-1.59105368

Evaluation on Test Data and Cost Reduction

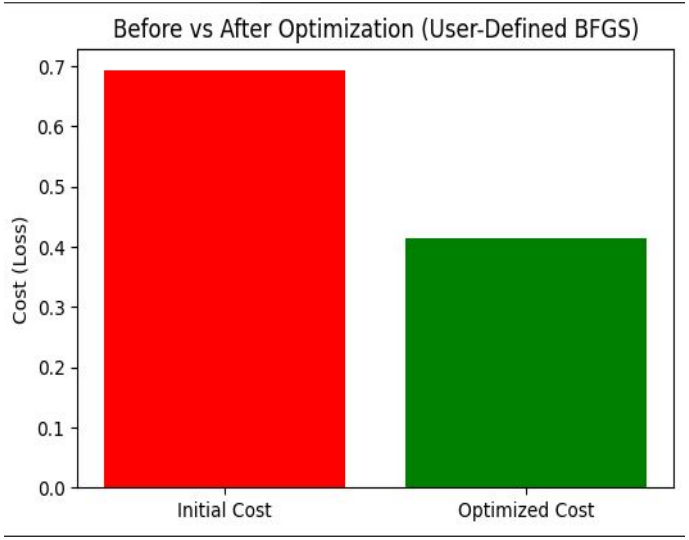
We are using the mean of the predicted probabilities as the threshold

Test Accuracy using BFGS(user-defined): 85.94%



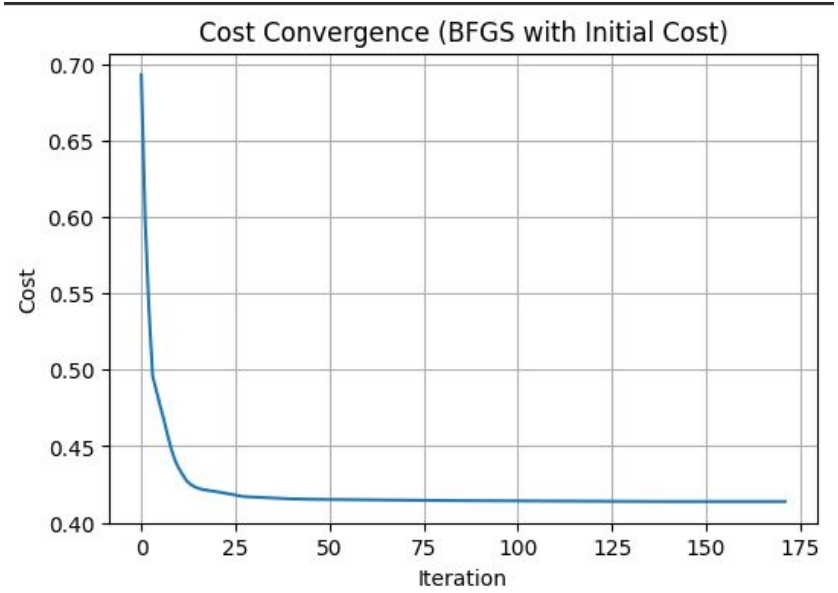
Cost Comparison:

	Metric	Value
0	Initial Cost	0.693147
1	Optimized Cost	0.413844
2	Cost Reduction	0.279304



Convergence Table

	Iteration	Cost	Alpha	Grad_Norm
0	0	0.693147	NaN	0.440821
1	1	0.599674	1.0	0.440821
2	2	0.541047	1.0	0.274554
3	3	0.496178	1.0	0.165274
4	4	0.486375	1.0	0.091960
166	166	0.413844	1.0	0.000029
167	167	0.413844	1.0	0.000027
168	168	0.413844	1.0	0.000023
169	169	0.413844	1.0	0.000020
170	170	0.413844	1.0	0.000018
171	171	0.413844	1.0	0.000012



The optimization algorithm converges for iteration number 171.

BFGS Method (Inbuilt)

1.Initial Values

$$\beta_0 = \beta_1 = \ldots\ldots = \beta_{30} = 0$$

$$Max_{Iteration} = 1000$$

$$Tolerance = 10^{-5}$$

2. Final Parameter Estimates

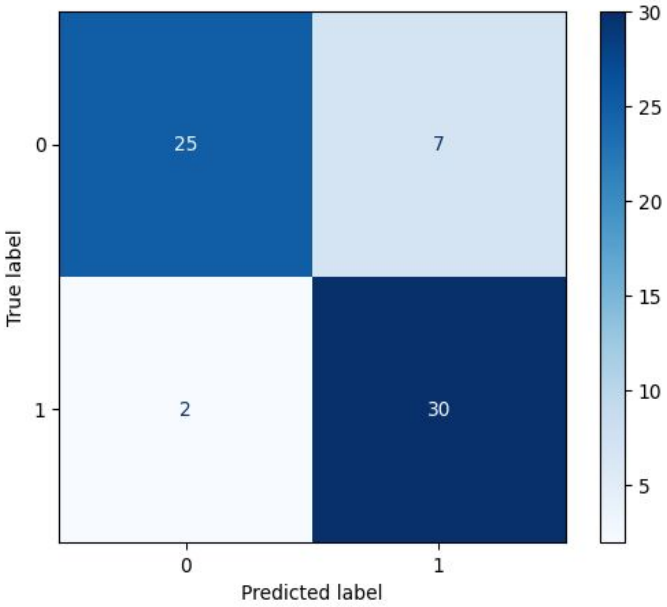
Parameter	Symbol	Value
Intercept	β_0 (Bias)	**-0.61241686**
Weight 1	β_1	-2.02727332
Weight 2	β_2	0.81504458
Weight 3	β_3	1.73517096
Weight 4	β_4	2.10691349
Weight 5	β_5	-0.22056934
Weight 6	β_6	0.94617558
Weight 7	β_7	-2.23441696
Weight 8	β_8	-0.08528783
Weight 9	β_9	-0.30560831
Weight 10	β_{10}	0.39152825
Weight 11	β_{11}	1.53775412
Weight 12	β_{12}	0.56869619
Weight 13	β_{13}	-0.07944537
Weight 14	β_{14}	-0.85393755
Weight 15	β_{15}	-0.82014875

Parameter	Symbol	Value
Weight 16	β_{16}	-0.03809811
Weight 17	β_{17}	-0.19316010
Weight 18	β_{18}	0.49853840
Weight 19	β_{19}	0.66186341
Weight 20	β_{20}	-0.27568773
Weight 21	β_{21}	-0.03429643
Weight 22	β_{22}	-0.17687947
Weight 23	β_{23}	0.51831466
Weight 24	β_{24}	-0.31101973
Weight 25	β_{25}	1.44449643
Weight 26	β_{26}	-2.82333046
Weight 27	β_{27}	0.04962031
Weight 28	β_{28}	-1.07901656
Weight 29	β_{29}	-1.82153711
Weight 30	β_{30}	-1.59105368

Evaluation on Test Data and Cost Reduction

We are using the mean of the predicted probabilities as the threshold.

Test Accuracy using BFGS: 85.94%

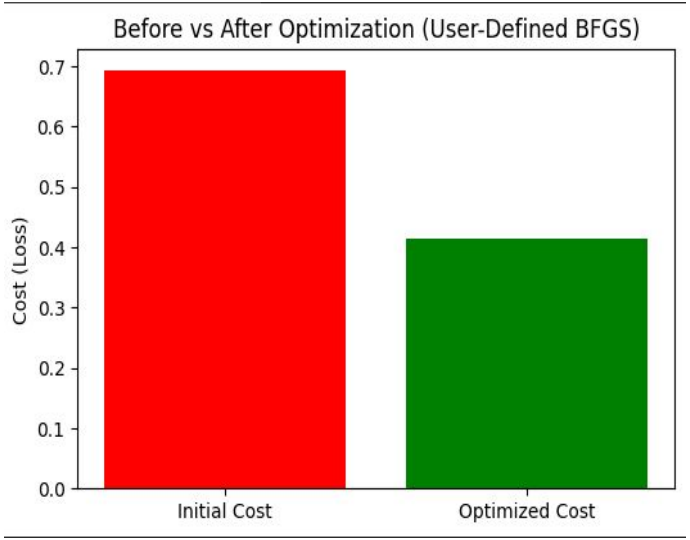


Cost Comparison:

	Metric	Value
0	Initial Cost	0.693147
1	Optimized Cost	0.413844
2	Cost Reduction	0.279304

Final cost value: 0.413844

Number of Iterations for convergence: 182



COMPARISON BETWEEN THE ALGORITHMS

COMPARISON CRITERIA	BFGS METHOD	STEEPEST DESCENT
Convergence Speed:	**Faster convergence, typically needing fewer iterations due to its approximation of the Hessian matrix.**	**Slow convergence, requiring many iterations, especially for ill-conditioned problems.**
Number of iterations and Cost Reduction	More expensive per iteration (due to matrix updates), but fewer iterations overall, often resulting in a lower total computational cost for smooth problems. (171 iterations for our problem) **Cost Reduction: 0.279304**	Cheap per iteration, but many iterations are needed. (3263 iterations for our problem). **Cost Reduction: 0.276613**
Efficiency	More efficient, with faster convergence often leading to lower overall computational expense despite the higher cost per iteration.	Less efficient for large or precise problems due to slow convergence.

Thank You